

ANCHOR: Asset-Informed Geometric Skills for Language-Conditioned Manipulation

Anonymous Authors

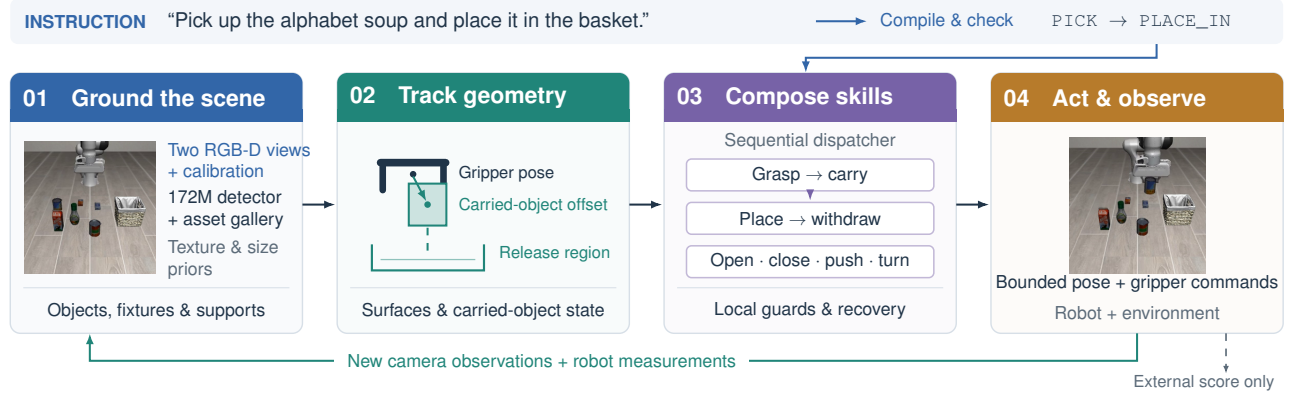


Fig. 1. ANCHOR uses a frozen 172M-parameter detector to ground objects and explicit geometric skills to control robot actions. Calibrated RGB-D and asset priors support each execution stage and its checks. Insets are upright frames from an actual Object 00 rollout; the geometry drawing is schematic. Benchmark success is recorded externally and is not fed back to the policy.

Abstract—Language-conditioned manipulation requires semantic grounding, geometric reasoning, and reliable transitions between physical interactions. ANCHOR uses a frozen 172M-parameter model to detect objects and geometric rules to control robot actions. It requires no robot demonstrations for action-policy training. Each execution step can be inspected, making failures easier to locate and fix. The system compiles supported instructions into grasp-place and contact skills using two calibrated RGB-D views and public asset priors. Explicit state tracks the carried object, and local checks manage release, articulation, and recovery. Benchmark success is recorded externally and is unavailable to the policy. A combined set of 400 task records, comprising 380 historical episodes and 20 targeted retest episodes, contains 374 successes (93.5%). It includes 3 controller versions; the latest controller has not been evaluated across all 40 tasks. Controller completion and external success disagree in 44 trials, demonstrating why these outcomes must be reported separately. Comparisons with published vision-language-action models describe model size, action-policy training, and task outcomes under the stated sensing and prior assumptions.

I. INTRODUCTION

Natural-language instructions provide a compact description of a manipulation goal, but leave much of the physical procedure unspecified. To place a bowl inside a drawer, a robot must identify the intended object, infer accessible geometry, establish a stable grasp, maintain clearance during transport, and release the object in a suitable region. Articulation introduces an additional difficulty: the robot must distinguish productive motion from mechanical resistance and decide when to release or reacquire contact. A plausible semantic interpretation alone does not resolve these questions.

Vision-language-action (VLA) policies offer one route to

this capability. OpenVLA [1] adapts pretrained representations to action prediction, while SmolVLA [2] studies a more compact architecture and efficient execution. Both learn robot actions from demonstrations. We study a system that uses pretrained perception to identify objects and computes actions from their geometry. This design avoids training an action network and makes the targets and transition checks of each skill available for inspection. Its requirements include calibrated depth, asset knowledge, and manually designed skills; these provide information that a learned policy may instead acquire from robot data.

We present ANCHOR (Fig. 1), an asset-informed baseline for language-conditioned tabletop manipulation. The system compiles supported instructions into ordered skill programs and binds their entities to geometry reconstructed from two calibrated RGB-D views. Its only neural model is a frozen 172M-parameter detector, used together with a gallery derived from public simulator assets. A dispatcher preserves state across grasp-place segments and invokes specialized contact controllers for drawers, doors, knobs, and pushing. Robot actions are bounded Cartesian pose increments with a gripper command.

The central engineering issue is continuity across physical stages. A semantically correct target can still be unreachable with the current wrist orientation; a successful close command need not establish retention; and a gripper placed at a receptacle center need not center the object being carried. ANCHOR therefore maintains explicit object-to-gripper displacement, re-observes relevant geometry at selected transitions, and uses local completion checks before advancing its program. These checks are fallible estimates, separate from benchmark success.

We evaluate the deployed implementation on the Spatial, Object, Goal, and Long suites of LIBERO [3]. Each suite contains ten evaluated tasks, with ten official initial states per task. The evaluation retains fixed task configurations and records controller versions; its scope is a system study on known benchmark assets and instruction forms. We make three contributions:

- 1) A language-to-geometry manipulation system that requires no robot demonstrations for action-policy training, with its sensing, asset priors, and manually designed skills explicitly specified.
- 2) A compact deployment using one frozen 172M-parameter detector, with model-size and task-outcome comparisons to representative VLAs on the four LIBERO suites.
- 3) Inspectable skill execution with explicit geometric targets and local checks, illustrated by targeted revisions to microwave and drawer execution that improve Long 09 and Goal 03 without updating neural weights.

The contribution is the integrated system and its empirical characterization. The targeted revisions are development results; their records and those of the original full evaluation retain their controller versions.

II. RELATED WORK

A. Language, Programs, and Geometric Planning

Task and motion planning combines discrete decisions with continuous geometric feasibility [4]. Language-conditioned systems extend this interface: SayCan connects language choices to available skills and their affordances [5], while Code as Policies generates executable programs that call perception and control interfaces [6]. ANCHOR uses a deterministic grammar and a fixed skill vocabulary. This makes the represented program inspectable, but restricts language coverage to implemented constructions and validated entities.

VoxPoser constructs spatial value maps to guide motion [7]. ReKep expresses manipulation objectives through relational keypoint constraints [8]. These works show the utility of intermediate geometric representations. Our implementation uses object and fixture geometry, carried-object offsets, and specialized finite-state controllers. It does not synthesize arbitrary constraint programs or perform a unified optimization over the complete task. Its behavior depends on the coverage and calibration of its manually implemented skills.

B. Pretrained Perception and Action Policies

Grounding DINO supports text-conditioned object detection [9]. ANCHOR uses a frozen local Tiny checkpoint as one source of region proposals; the downstream geometric and gallery components remain essential to the implemented pipeline. In particular, public object textures and collision dimensions provide priors that are stronger than an image-only input setting.

OpenVLA [1] and SmolVLA [2] learn action generation from robot data. OpenVLA-OFT [10] studies action chunking, parallel decoding, continuous action prediction, and additional sensor inputs. ANCHOR provides a complementary, explicitly engineered control baseline. The distinction concerns how actions are obtained; it does not imply an absence of learning in the pretrained detector or an absence of development on benchmark scenarios. Section V-A compares model size, policy-training requirements, and reported success rates while identifying differences in sensing, asset access, and evaluation conditions.

C. Benchmark Interpretation

LIBERO was introduced to study knowledge transfer in lifelong robot learning [3]. We use its four commonly evaluated ten-task suites to characterize a fixed manipulation system, rather than measuring continual learning or forgetting. The choice of ten trials per task follows the sample count used in SmolVLA’s simulation study [2]. Matching that count alone does not reproduce its training or evaluation conditions. We specify the remaining conditions in Section IV.

III. SYSTEM

A. Observation and Prior Information

At step t , the policy receives

$$o_t = (\{I_t^c, D_t^c, K^c, T_{WC,t}^c\}_{c \in \{a,w\}}, r_t), \quad (1)$$

where a and w denote the fixed and wrist cameras, I is RGB, D is metric depth, K is calibration, and T_{WC} maps camera coordinates into the world frame. Robot measurements r_t include end-effector pose, joint state, gripper state, and available force/torque measurements. The grasp-place adapter uses pose, joint positions, and gripper width; contact execution can access the broader robot measurement interface.

These are simulated, calibrated measurements. In particular, camera extrinsics and proprioception are supplied by the simulator adapter, and depth is not an estimate from monocular RGB. The policy observation types contain no object pose labels, segmentation IDs, BDDL predicates, reward, or task-success flag. Task identifiers and initial-state indices remain with the evaluator; reset passes the instruction and a nonsemantic episode token. This is a software interface property supported by code inspection, not a claim of adversarial isolation.

Offline information is represented by a prior collection $\mathcal{P}$. It includes object texture histograms, dimensions derived from public collision geometry, robot geometry, class-specific handling rules, and motion thresholds. The evaluated controller is therefore $a_t = \pi(o_{\leq t}, \ell; \mathcal{P})$, where ℓ is the instruction. No action-policy fitting or demonstration loading occurs in this evaluation. The detector’s pretraining and the controller’s manual development remain part of the system’s knowledge.

B. Constrained Language Compilation

The compiler maps supported language to

$$\mathcal{G}(\ell) = (s_1, \dots, s_M), \quad s_j = (k_j, e_j^{\text{src}}, e_j^{\text{dst}}, \rho_j). \quad (2)$$

Here k_j is a skill, the entity bindings specify source and destination, and ρ_j represents a placement relation or instance selector. The vocabulary contains pick, place-on, place-in, relative placement, stack, open, close, push, turn-on, and turn-off. For example, a two-object basket instruction becomes two successive pick/place pairs. A drawer instruction can produce an articulation segment followed by a manipulation segment.

Bindings retain relations such as *next to*, *between*, or an ordering among visible instances. They do not encode the simulator’s object identity. The complete program undergoes capability checks before motion, including valid skill arguments and consistency of held-object state. Compilation success only establishes representational support; it does not establish physical executability. The deployed grammar is tailored to known LIBERO phrases and an allow-listed entity vocabulary. An optional semantic-resolver interface exists, but no language-model resolver is configured in this run.

C. Visual Grounding and Geometric State

The deployed Grounding DINO Tiny checkpoint contains 172,249,090 unique parameters, including its visual and text encoders. All weights are frozen, and one detector instance serves both camera views in each evaluation process. The asset gallery, language compiler, and geometric controllers add no neural model parameters. The pinned checkpoint and a reproducible parameter count are recorded with the manuscript artifacts.

Each calibrated pixel $\tilde{u} = (u, v, 1)^\top$ with valid depth contributes

$$x_W = R_{WC}(D(u, v)K^{-1}\tilde{u}) + p_{WC}. \quad (3)$$

Workspace filtering, support estimation, and connected components yield candidate point sets. The pipeline fits observed bounds and uses texture/size priors to classify and complete partial observations. The gallery computes a color similarity

$$b(h, h_k) = \sum_i \sqrt{h_i h_{k,i}} \quad (4)$$

between normalized HSV histograms. Its score combines color and shape terms, with configured weights 0.4 and 0.6, a softmax temperature of 0.13, and additional class-dependent footprint gates. Grounding DINO supplies language-conditioned boxes that are converted to 3-D hypotheses using depth. Thus a detector box is a proposal, rather than an oracle object mask.

Cross-camera hypotheses with matching labels are clustered in 3-D. For a cluster, the implemented fusion weights are proportional to $\max(q_i, 0.05)\sqrt{n_i}$, where q_i is confidence and n_i is point count. The default center-distance threshold is 4.5 cm. These thresholds are empirical operating choices; nearby repeated objects can still produce ambiguous associations.

The state retains both observed support/bounds and completed geometry. This distinction matters when an object is partially visible: a size prior can stabilize a center estimate, but should not be mistaken for measured free space. Geometric selectors resolve the intended instance from the available candidates and reference objects. At selected phase transitions, new views are requested and checked against prior observations to reduce hand-occlusion-induced jumps. Specialized fixture and cavity routines refine support surfaces, opening regions, and admissible approach geometry.

D. Grasping, Transport, and Placement

Consecutive manipulation skills share a grasp–place controller. Its phases include acquisition, pregrasp motion, approach, finger closure, retention checking, lift, transport, descent, release, and retreat. Transitions combine pose tolerances, measured gripper width, visual evidence, and bounded retry logic. Different grasp modes account for exterior pinch, rim pinch, and interior expansion; mode choice depends on the object and observed accessibility. These are mechanical templates with specialized branches.

For target pose (p^*, R^*) , the basic servo emits

$$a_t = \text{clip}_{[-1,1]} \begin{bmatrix} (p^* - p_t)/s_p \\ \text{Log}(R^* R_t^\top)^\vee / s_R \\ g_t \end{bmatrix}. \quad (5)$$

The general grasp–place defaults are $s_p = 0.05$ m and $s_R = 0.25$ rad; cavity handling can select another rotation scale. The nominal pose tolerances are 8 mm and 0.12 rad. These are normalization scales and transition tolerances, not measured tracking-error bounds or a continuous-time stability guarantee.

The gripper and carried object generally have different centers. Let $\hat{d}_t$ be the estimated world-frame vector from the end effector to the carried-object reference point. For desired object location p_o^* , placement planning uses the geometric relation

$$p_{ee}^* = p_o^* - \hat{d}^*, \quad \hat{d}^* = \Delta R \hat{d}_t \quad (6)$$

when the planned wrist motion rotates the held geometry by ΔR . The implementation adds support, release-height, aperture, and fixture-specific corrections to this relation. Offset estimates may come from precontact geometry or refreshed RGB-D evidence; their origin is retained in the execution trace. This allows inspection of placement decisions without assuming that all offset estimates are equally reliable.

Release is followed by sensor-based verification. A local geometric check can reject a placement even when the benchmark predicate has already been true, or accept a relation that the benchmark does not recognize. Section V-D quantifies this distinction rather than using controller completion as the reported success metric.

E. Contact Skills and Sequential Execution

The dispatcher separates consecutive grasp–place steps from atomic contact skills. Drawer and door interactions use observed fixture geometry to construct approach, engagement, motion, release, and retreat stages. Knob and push

skills use their own target detectors and motion templates. Shared episode state includes an initial robot-pose reference and, where applicable, an observed drawer anchor for subsequent placement.

Contact evidence combines position error, lack of measured progress, and changes in end-effector force. One implemented guard requires a minimum contact duration and bounded position error, together with either a stalled error reduction or sufficient force change. Such a guard indicates a candidate contact condition; it is not the task predicate. Specialized post-contact visual checks assess the resulting fixture configuration.

Local recovery is bounded by phase counters and geometric checks. Examples include releasing a rejected grasp, reacquiring a visible handle, or performing a short clearance motion after stalled drawer interaction. These routines preserve the distinction between a robot-side mechanical signal and an estimate of task completion. Their hand-selected parameters are part of the fixed controller configuration evaluated below.

Several fixture skills refine the grounded object box into physical contact geometry. A complete visible burner disk anchors the stove support centre; public plate thickness supplies the offset from the visible disk to the load-bearing surface. That unoccluded estimate is retained across sequential placements. For an open microwave, visible vertical control-panel and side-wall surfaces, together with the roof height, constrain the cavity and hinge frame through public physical dimensions. A flat tabletop crop or a roof without supporting vertical structure is insufficient for this fit.

Placement also accounts for the hand’s withdrawal. The microwave approach aligns the payload with the cavity width and uses a lateral entry path. Repeated basket placements receive separate diagonal landing centres. Before withdrawing from a basket, the controller opens the jaws to a measured clearance; a bounded open-wrist tilt clears tall bodies near the palm. All such proposals use language, RGB-D estimates, and robot measurements; the external task predicate is unavailable to their selection or execution.

The revised microwave close skill fits a vertical door slab within the sensor-grounded hinge sweep. Initial fitting excludes the fixed front frame by requiring panel evidence outside the appliance body. A compact top-down gripper approaches the exterior face and follows bounded hinge-frame waypoints. After a motion plateau, a fresh panel-angle estimate can initiate withdrawal; a second view after retreat verifies closure. Placement remains separate and now requires a measured 78-mm jaw opening and a settling dwell before withdrawing from the cavity, preventing a mug rim from remaining hooked on a finger. A closed door alone need not satisfy the full task.

IV. EXPERIMENTAL PROTOCOL

A. Tasks, Initial States, and Runtime

The evaluation contains $4 \times 10 \times 10 = 400$ episodes: Spatial, Object, Goal, and Long, ten tasks each, and official initial-state indices 0–9. Long is named `libero_10` in

TABLE I
DECLARED CONDITIONS FOR THE EVALUATED SYSTEM.

Property	Setting
Policy observations	Fixed and wrist RGB-D, camera calibration, robot measurements
Offline priors	Public asset textures/dimensions, robot geometry, hand-written skill rules
Action generation	Geometric state machines and clipped Cartesian increments
Neural model	One frozen Grounding DINO Tiny (172M parameters)
Action-policy training	None; skills and thresholds are designed manually
Scene truth in policy	No object pose labels, segmentation IDs, or BDDL predicates in observation types
Success feedback	External latch; no score-driven policy stopping
Coverage	Known benchmark assets and implemented instruction grammar

the implementation. Global random generators and the environment configuration use seed 7. The environment adapter preserves or replays the task reset stream when starting at a later initial-state index, because resetting can affect fixture configuration beyond the saved joint state.

We use two 256×256 RGB-D cameras and a Panda with operational-space pose control. A ten-step settling period precedes scored actions. The Spatial/Object/Goal/Long action budgets are 220/280/300/520, matching the numerical budgets in the OpenVLA evaluation script [11]. Our environment seed and success-handling logic differ from that script; its environment utility uses seed 0. The original full run uses Ubuntu 22.04, Python 3.12, PyTorch 2.11.0 with CUDA 12.8, MuJoCo 3.3.2 [12], robosuite 1.4.0 [13], and hf-libero 0.1.4 on one RTX 5090. The targeted retests use the same software environment on an RTX 4090.

The original full campaign records 360/400 successes with one frozen controller. We subsequently revise Long 09 microwave execution, then Goal 03 drawer execution. Each task is retested on all ten official initializations, in index order, with one frozen source snapshot. The retests record 8/10 and 9/10 successes, respectively. The combined statistics retain 380 original episodes for the other 38 tasks and replace both complete task records, including failures. They contain 3 controller versions and do not establish a full-suite result for the latest controller. The original campaign, all source manifests, and each record’s source version are preserved. These initializations informed development; the retests are not held out.

B. External Scoring and Internal Completion

Let q_t be the official success predicate observed by the evaluator. With policy-controlled stopping time τ capped at budget H , the recorded outcome is

$$Y = \bigvee_{t=1}^{\tau} q_t \vee q_{\text{final}}, \quad \tau \leq H. \quad (7)$$

The final query also handles a zero-action stop. Reward and success do not enter the policy observation. The evaluator continues after a success signal until the controller requests

TABLE II

OBSERVED SUCCESS AND EXECUTION COST. SR USES EXTERNAL EVER-SUCCESS. THE COMBINED RECORDS RETAIN 380 HISTORICAL EPISODES AND USE 20 TARGETED RETEST EPISODES. [†]MEDIANS POOL SOURCE-RUN ELAPSED TIMES; HARDWARE AND CONCURRENCY DIFFER.

Suite	Success	SR (%)	Mean steps	Median $s^{\dagger}$
Spatial	93/100	93.0	147.1	68.1
Object	98/100	98.0	195.3	59.5
Goal	97/100	97.0	179.9	66.2
Long	86/100	86.0	324.5	93.4
Overall	374/400	93.5	211.7	68.1

stopping or the action budget expires. The score therefore represents *ever reaching* the predicate during this rollout, not necessarily satisfying it at the final frame.

Each record additionally stores terminal controller state, step count, elapsed time, failure message, and geometric traces. The reported success rate is $\hat{p} = N^{-1} \sum_i Y_i$. Equal task sample counts make the overall episode average equal to the macro-average over tasks in the completed run. Each reported batch retains all planned failures; candidate outcomes are not mixed within a task. Controller-internal retries remain within one episode.

C. Scope of the Evidence

All three source runs pass their respective coverage checks. The combined record set contains 400 unique task/initial-state pairs with the declared configuration; original record hashes identify the retained and replaced episodes independently. Tables are generated from hashed original records. Published values are stored separately with their cited sources. Per-task counts expose the sample size behind each rate.

V. RESULTS AND ANALYSIS

A. Comparison with Published Policy Results

Table III reports neural model size and whether robot demonstrations are used to train or fine-tune action generation. ANCHOR uses 0.172B parameters in one frozen detector and has no learned action network. Its FP32 weight file occupies 689 MB (decimal). The selected SmolVLA model is about 2.6 times larger, and the reported 7B OpenVLA model class is about 40.6 times larger. These comparisons count the full detector, including its text encoder. Published VLA sizes are the source descriptions, not new counts of their deployed checkpoints.

Robot-demonstration training and general pretraining are distinct: ANCHOR relies on pretrained perception and manually developed skills. The selected SmolVLA simulation model starts from a VLM without robotics pretraining, but still trains its action policy on LIBERO demonstrations [2]. Thus the table’s training column concerns learning robot actions, not whether a model has ever been pretrained.

Table IV compares our measurements with published LIBERO success rates, retaining the original papers’ reported

TABLE III

MODEL SIZE AND ROBOT-DEMONSTRATION USE FOR ACTION-POLICY TRAINING. VLA SIZES ARE ROUNDED VALUES FROM THE CITED PAPERS; ANCHOR COUNTS ALL UNIQUE PARAMETERS IN ITS DEPLOYED DETECTOR. “NONE” REFERS TO ACTION-POLICY TRAINING, NOT TO PERCEPTION PRETRAINING OR MANUAL SKILL DEVELOPMENT.

Method	Parameters	Policy training
SmolVLA [2]	0.45B	Robot demos
OpenVLA [1]	7B	Robot demos
OpenVLA-OFT [10]	7B	Robot demos
ANCHOR	0.172B	None

averages. Our implementation is the only system evaluated in the present study.

The table specifies the distinct sensing and policy-training settings. The OpenVLA study uses filtered demonstrations [1]. SmolVLA trains across all 40 tasks and evaluates ten trials per task [2]. The selected OpenVLA-OFT setting uses filtered demonstrations and evaluates 500 trials per suite [10]. ANCHOR uses calibrated RGB-D, asset priors, and the scoring rule in Eq. (7). These differences make the table a comparison of reported system outcomes under their stated conditions, rather than a controlled ranking.

OpenVLA-OFT reports a higher average success rate than our combined record set. Our results support the value of a smaller neural component and the absence of demonstration-based action training in this declared setting; they do not establish a success-rate advantage over all VLAs. Long has the lowest success rate for every listed method, including ANCHOR, motivating a closer examination of multi-stage failures.

B. Aggregate and Task-Level Performance

The combined records contain 374 successes in 400 episodes (93.5%; Table II). Suite success rates are 93.0% for Spatial, 98.0% for Object, 97.0% for Goal, and 86.0% for Long.

Figure 2 shows the task-level distribution. 24 tasks have ten successful recorded trials. Long 09, placing a mug inside a microwave and closing it, succeeds in 8/10 trials. Long 08, placing both moka pots on the stove, succeeds in 7/10 trials. These tasks combine placement with confined access, shared supports, or fixture interaction.

Drawer tasks show varied outcomes: Goal task 03, opening the top drawer and placing a bowl inside, succeeds in 9/10 trials; Spatial task 04, retrieving a bowl from a top drawer, succeeds in 7/10; and Long task 03, placing a bowl in a bottom drawer and closing it, succeeds in 7/10. These complete task records identify difficult scenarios and stopping reasons; they do not establish articulation as the cause of failure.

C. Inspecting and Revising Failed Skills

Long 09 separates mug placement, withdrawal, door contact, and closure verification. Inspection found a mug rim remaining hooked on a finger. The revision checks a measured

TABLE IV

LIBERO SUCCESS RATES (%). PUBLISHED ROWS CONTAIN THE REPORTED MEANS FROM OPENVLA TABLE 12 [1], SMOLVLA TABLE 2 [2], AND OPENVLA-OFT TABLE I [10]; THEY ARE NOT RERUNS BY US. OPENVLA-OFT USES ITS FULL SETTING. THE FINAL ROW IS MEASURED IN THIS WORK. IT COMBINES 380 HISTORICAL EPISODES AND 20 TARGETED RETEST EPISODES FROM 3 CONTROLLER VERSIONS; IT IS NOT A FULL RERUN OF THE LATEST CONTROLLER. ALL METHODS RECEIVE LANGUAGE; THEIR SENSING, TRAINING, AND PROTOCOLS DIFFER.

Method	Sensing and action-policy setup	Spatial	Object	Goal	Long	Average
<i>Published results (reported means)</i>						
Diffusion Policy [1]	Single RGB; policy trained from scratch per suite	78.3	92.5	68.3	50.5	72.4
Octo [1]	Single RGB; pretrained policy fine-tuned per suite	78.9	85.7	84.6	51.1	75.1
OpenVLA [1]	Single RGB; pretrained policy fine-tuned per suite	84.7	88.4	79.2	53.7	76.5
SmolVLA (0.45B) [2]	RGB/state; VLM-initialized policy trained across suites	90.0	96.0	92.0	71.0	87.3
OpenVLA-OFT [10]	Two RGB views + state; policy fine-tuned per suite	97.6	98.4	97.9	94.5	97.1
<i>Measured in this work</i>						
ANCHOR	Two RGB-D views + state; asset priors, geometric skills	93.0	98.0	97.0	86.0	93.5

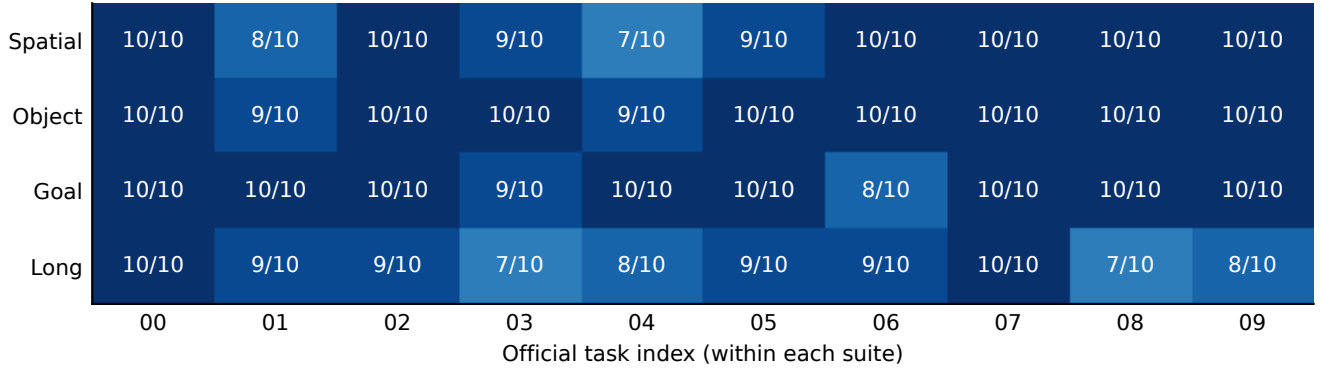


Fig. 2. External successes per ten trials. Columns identify different tasks in each suite; darker cells indicate higher success. Goal 03 and Long 09 use complete targeted retests; other cells retain the original full-campaign records.

78-mm jaw opening and waits for settling; closure estimates the visible door panel and pushes its exterior face through bounded hinge-frame waypoints. A complete retest records 8/10 successes, compared with 0/10 originally.

Goal 03 originally records 3/10. Traces expose excessive motion after handle release, including wrist and elbow joint-limit encounters. The revision compares equivalent jaw frames using continuous public-model IK and bounds simultaneous translation and rotation. It also moves a held bowl outside the observed drawer front before lifting, to avoid the lip. A complete retest records 9/10. Both revisions preserve the detector weights, official initializations, action budgets, and scoring. These development cases show how explicit geometry and stages support localized code changes; remaining failures are retained in the records.

D. Controller Completion Is a Different Outcome

Table V crosses terminal controller state with external success. In 7 episodes the controller reports completion while the evaluator reports failure. Conversely, 37 episodes receive external success without controller completion. The combined disagreement is 44/400 (11.0%). The latter group includes cases where a predicate is reached before a later local failure or budget exhaustion. Because the recorder retains an ever-success latch, these records do not imply successful final-state verification.

TABLE V

TERMINAL CONTROLLER STATE AND INDEPENDENTLY RECORDED EXTERNAL EVER-SUCCESS. A CONTROLLER FAILURE CAN FOLLOW AN EARLIER BENCHMARK SUCCESS.

Controller terminal state	External success	External failure
Succeeded	337	7
Failed	26	19
Timeout	11	0

Using local completion as the benchmark label would change the measured outcome; feeding external success to the controller would change its information interface. We record both values separately. The records lack first-success time and a separate final-only label, so they cannot quantify changes in final-state success from continued execution or the runtime saved by an early-stop evaluator.

E. Recorded Failure Categories

Table VI groups the 26 externally unsuccessful episodes using their persisted status and failure message. The grouping gives controller completion/external failure and global-budget exhaustion precedence; remaining messages are classified by phase timeout, grasp/retention rejection, sensor-verification rejection, or another failure. This is a repro-

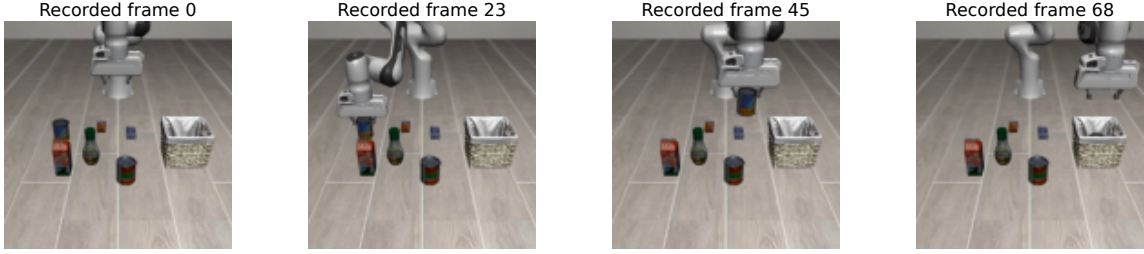
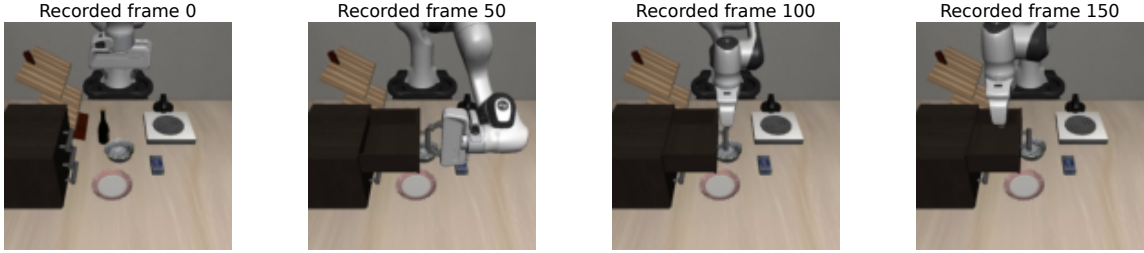
Object 00 / init 00: external success**Goal 03 / init 02: action-budget exhaustion**

Fig. 3. Actual fixed-camera frames. Top: *pick up the alphabet soup and place it in the basket* (Object 00, initial state 00). Bottom: *open the top drawer and put the bowl inside* (Goal 03, initial state 02). Both episodes come from the original full campaign, before the targeted revisions. Four uniformly spaced recorder indices are shown (two-action recording stride). Frames are vertically flipped for upright display; left/right is preserved.

TABLE VI

OUTCOME GROUPS AMONG EXTERNAL FAILURES. CATEGORIES ARE DERIVED FROM RECORDED CONTROLLER MESSAGES WITH A FIXED PRECEDENCE RULE.

Recorded outcome group	Episodes
Phase timeout	13
Controller completion / external failure	7
Grasp / retention rejection	3
Other controller failure	2
Sensor verification rejection	1

ducible description of the recorded stopping reason, not a manually verified taxonomy of physical root causes.

A phase timeout may arise from an inaccessible target, calibration error, occlusion, or contact resistance. Likewise, a rejected sensor verification may reflect an actual placement error or perception uncertainty. Determining these causes requires frame-level or geometric-trace analysis.

F. Execution Cost and Qualitative Context

The median episode across the targeted retests takes 69.6 seconds. Mean action counts range across suites as shown in Table II. Long episodes average 324.5 actions, compared with 195.3 for Object. These costs include controller execution through its own stopping condition; they are not time-to-first-success. Recorded elapsed times mix perception, simulation, rendering, and video overhead under each run’s hardware and scheduling configuration. They should not be interpreted as real-robot control frequency.

The run reports 0 runtime exceptions. The targeted retest dispatcher durations sum to 24.5 minutes. There is no single wall-time measurement for the combined 400-episode record

set. Figure 3 provides visual context from actual recordings. The upper row shows a successful Object task-00 episode; the lower row is an unsuccessful Goal task-03 initialization in the original campaign. These historical examples illustrate execution, not current failure prevalence.

VI. DISCUSSION AND LIMITATIONS

The evaluated system combines a small neural model, no demonstration-based action-policy training, and inspectable execution stages. These properties depend on calibrated RGB-D, object priors, a constrained grammar, and manually developed skills. They describe a useful operating point for known environments, with task performance and development effort both dependent on the coverage of those skills.

a) Resources and development effort.: The parameter count and weight-file size quantify the neural component. They do not establish proportional savings in end-to-end latency or peak memory, which also depend on perception processing, control, and simulation. Avoiding action-policy fitting does not remove detector pretraining or the engineering effort required to build and revise geometric skills. The recorded stages support inspection, but debugging effort was not measured against a learned policy.

b) Generalization and novelty.: The run covers known assets, language forms, and one simulated embodiment with calibrated depth. It does not evaluate transfer to unseen assets, language, perturbed calibration, or physical hardware, and provides no held-out history of controller development. The rates characterize the deployed benchmark system. Its geometric operations and state machines are established techniques; the contribution is their documented integration and empirical characterization.

c) *Reliability and interpretation.*: Bounded actions and local guards limit the implemented commands, but do not establish collision safety, contact-force safety, or real-world reliability. The narrow ten-trial task samples can also conceal rare failure modes. The observed controller/evaluator disagreement points to a specific measurement improvement: record first-success time and final predicate separately, then evaluate alternative stopping conventions without silently changing the policy’s observations.

d) *Reproducibility.*: Source and resource checksums, fixed initial-state identifiers, and record-derived figures support reconstruction of this evaluation. Boundary counters are implementation-generated records, not independent attestations.

VII. CONCLUSION

ANCHOR uses a frozen 172M-parameter model to detect objects and geometric rules to control robot actions. It requires no robot demonstrations for action-policy training, and its explicit execution steps support inspection and targeted repair. Under the declared sensing and asset assumptions, the combined records contain 93.5% external successes across 400 episodes, with Long 09 at 8/10 and Goal 03 at 9/10. The combined records contain 3 controller versions; broader performance of the latest controller remains unmeasured. The study documents model requirements, task outcomes, and targeted skill revisions while preserving the distinction between local completion and external success.

ACKNOWLEDGMENTS AND AI DISCLOSURE

OpenAI GPT-6-Astra (via Codex) assisted in drafting and editing the text in all sections, preparing the LaTeX source, and writing scripts for the numerical tables and figures. Results for ANCHOR are computed from existing simulator evaluation records; comparison values are taken from the cited publications. The rollout images are extracted from our recordings; no generative image model was used.

REFERENCES

- [1] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn, “OpenVLA: An open-source vision-language-action model,” *arXiv preprint arXiv:2406.09246*, 2024, version 3, Appendix E, Table 12.
- [2] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zouitine, M. Aractingi, C. Pascal, M. Russi, A. Marafioti, S. Alibert, M. Cord, T. Wolf, and R. Cadene, “SmolVLA: A vision-language-action model for affordable and efficient robotics,” *arXiv preprint arXiv:2506.01844*, 2025, version 1, Table 2.
- [3] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, “LIBERO: Benchmarking knowledge transfer for lifelong robot learning,” *arXiv preprint arXiv:2306.03310*, 2023.
- [4] C. R. Garrett, R. Chitnis, R. Holladay, B. Kim, T. Silver, L. P. Kaelbling, and T. Lozano-Pérez, “Integrated task and motion planning,” *arXiv preprint arXiv:2010.01083*, 2020.
- [5] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, *et al.*, “Do as i can, not as i say: Grounding language in robotic affordances,” *arXiv preprint arXiv:2204.01691*, 2022.
- [6] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng, “Code as policies: Language model programs for embodied control,” *arXiv preprint arXiv:2209.07753*, 2022.
- [7] W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, and L. Fei-Fei, “VoxPoser: Composable 3d value maps for robotic manipulation with language models,” *arXiv preprint arXiv:2307.05973*, 2023.
- [8] W. Huang, C. Wang, Y. Li, R. Zhang, and L. Fei-Fei, “ReKep: Spatio-temporal reasoning of relational keypoint constraints for robotic manipulation,” *arXiv preprint arXiv:2409.01652*, 2024.
- [9] S. Liu, Z. Zeng, T. Ren, F. Li, H. Zhang, J. Yang, Q. Jiang, C. Li, J. Yang, H. Su, J. Zhu, and L. Zhang, “Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection,” in *European Conference on Computer Vision*, 2024.
- [10] M. J. Kim, C. Finn, and P. Liang, “Fine-tuning vision-language-action models: Optimizing speed and success,” *arXiv preprint arXiv:2502.19645*, 2025, version 2, Table I.
- [11] OpenVLA Contributors, “LIBERO evaluation script and environment utilities,” <https://github.com/openvla/openvla>, accessed September 2026. Paths: `experiments/robot/libero/run_libero_eval.py` and `libero_utils.py`.
- [12] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A physics engine for model-based control,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2012, pp. 5026–5033.
- [13] Y. Zhu, J. Wong, A. Mandlekar, R. Martín-Martín, A. Joshi, K. Lin, A. Maddukuri, S. Nasiriany, and Y. Zhu, “robosuite: A modular simulation framework and benchmark for robot learning,” *arXiv preprint arXiv:2009.12293v3*, 2025.